

4 Программирование

Все запросы выполнялись в СУБД PostgreSQL, развёрнутой в контейнере Docker на машине Apple MacBook Air с процессором Apple M3 и 16 ГБ оперативной памяти. База данных содержит 317 614 записей в 19 таблицах; основные транзакционные таблицы — `submission` (117 043 строк), `participation` (58 631 строк) и `team_member` (58 228 строк); справочные таблицы содержат от 4 до 10 строк. Полный состав таблиц и число строк в каждой из них приведены в таблице 20.

Таблица 20: Число строк в таблицах базы данных

Таблица	Число строк
<code>submission</code>	117 043
<code>participation</code>	58 631
<code>team_member</code>	58 228
<code>user</code>	32 350
<code>team</code>	19 498
<code>competition_dataset</code>	11 386
<code>competition</code>	7 567
<code>configuration</code>	7 567
<code>dataset_file</code>	3 963
<code>dataset</code>	1 321
<code>role</code>	10
<code>competition_status</code>	8
<code>dataset_purpose</code>	8
<code>submission_status</code>	8
<code>participation_status</code>	6
<code>task_type</code>	6
<code>team_status</code>	6
<code>metric</code>	4
<code>team_role</code>	4
Итого	317 614

Визуализации планов выполнения запросов были построены с помощью сервиса explain.dalibo.com [1]. Он позволяет загрузить результат команды `EXPLAIN ANALYZE` и отображает план запроса в виде дерева.

Каждый узел плана содержит параметр `cost`, который показывает оценочную стоимость выполнения операции. Значение записывается в формате `cost=<start>..<total>`, где `start` обозначает стоимость получения первой строки, а `total` — полную стоимость выполнения узла.

Планировщик PostgreSQL вычисляет стоимость до выполнения запроса. Чем больше значение `cost`, тем более ресурсоёмкой считается операция. Стоимость корневого узла соответствует общей стоимости всего плана запроса.

В интерфейсе explain.dalibo.com узлы дополнительно выделяются цветом: дешёвые операции отображаются синим цветом, а дорогие — красным. Это упрощает поиск узких мест в запросе.

4.1 Запрос 1.1

Найти соревнования, в которых участвовал пользователь *A* в роли *B*, и команда которого отправила хотя бы одну попытку с файлом датасета *C*.

```
1 SELECT DISTINCT
2     c.title      AS comp_title,
3     u.username   AS user_name,
4     r.name       AS user_role,
5     df.filename  AS file_name
6 FROM competition c
7 JOIN participation p
8     ON p.competition_id = c.competition_id
9 JOIN team_member tm
10    ON tm.team_id       = p.team_id
11 JOIN "user" u
12    ON u.user_id        = tm.user_id
13 JOIN role r
14    ON r.role_id        = u.role_id
15 JOIN submission s
16    ON s.participation_id = p.participation_id
17 JOIN competition_dataset cd
18    ON cd.competition_id = c.competition_id
19 JOIN dataset d
20    ON d.dataset_id      = cd.dataset_id
21 JOIN dataset_file df
22    ON df.dataset_id     = d.dataset_id
23 WHERE
24     u.username = $1
25 AND r.name = $2
26 AND d.name = $3
27 ORDER BY
28     c.title;
```

Планировщик начинает с точечного поиска по индексам: **user** фильтруется по **username** через **user_username_key** (cost 0.31), **role** — по **name** через Seq Scan (cost 1., таблица мала). Участники команды раскрываются через Bitmap Heap Scan по **team_member** (cost 4.20), затем Nested Loop по индексу **participation** связывает их с соревнованиями. Наиболее дорогой узел плана — Seq Scan по **dataset_file** (cost 977.60): таблица не фильтруется на этом шаге и читается целиком перед Hash Join с **dataset** (cost 14.32). Цепочка **competition_dataset** и **submission** обходится дёшево благодаря Index Scan по первичным ключам (cost 0.02 и 0.48 соответственно). Финальные Sort и Unique имеют суммарный cost 0.01, поскольку к этому моменту набор строк уже минимален. **DISTINCT** устраняет дубликаты, возникающие из-за нескольких файлов одного датасета и нескольких попыток одного участия.

Реляционная формула запроса:

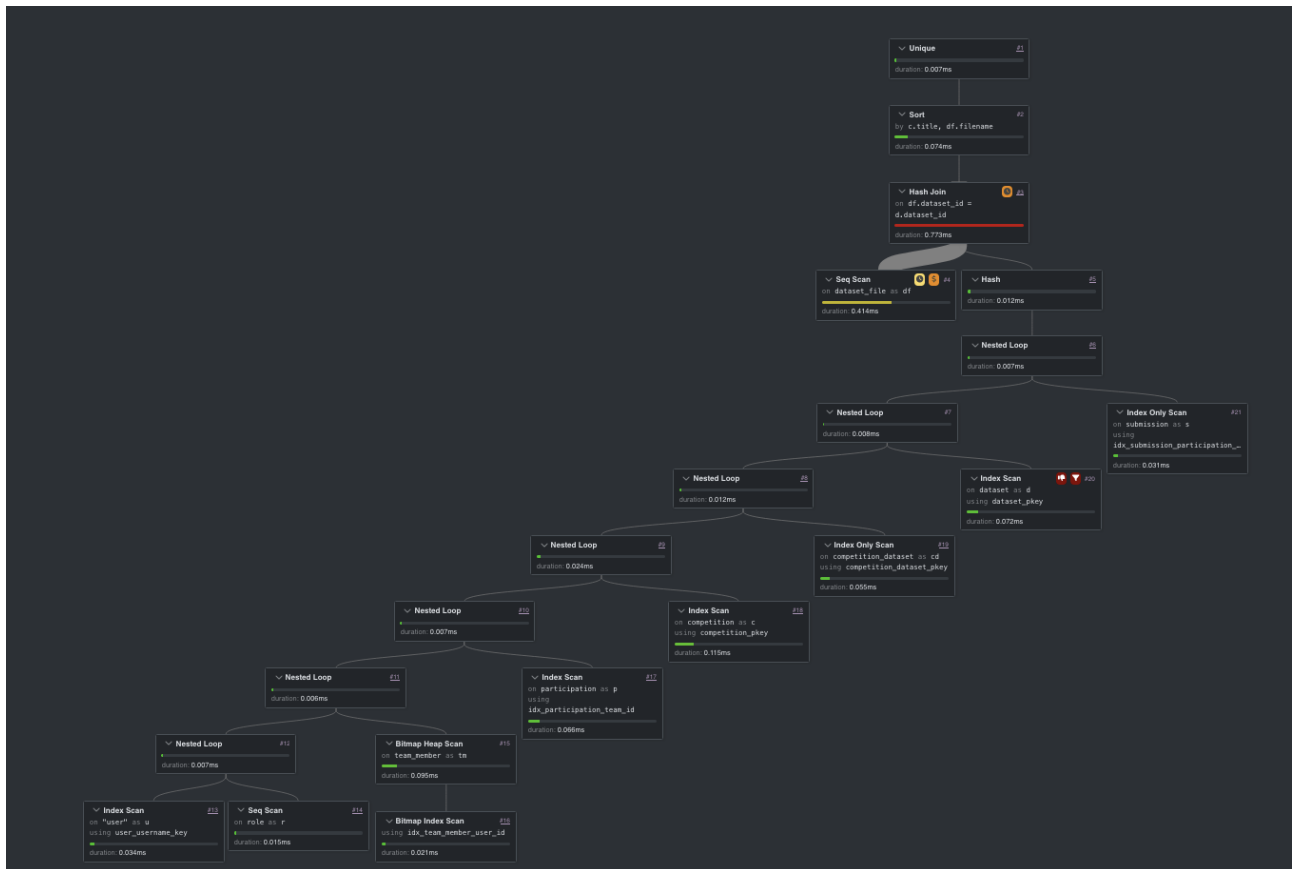
$$\pi_{\text{title, username, role, filename}} \left(\sigma_{\text{username}=A \wedge \text{role}=B \wedge \text{dataset}=C} \left(\begin{aligned} &\text{competition} \\ &\bowtie_{\text{competition_id}} \text{participation} \\ &\bowtie_{\text{team_id}} \text{team_member} \\ &\bowtie_{\text{user_id}} \text{user} \\ &\bowtie_{\text{role_id}} \text{role} \\ &\bowtie_{\text{participation_id}} \text{submission} \\ &\bowtie_{\text{competition_id}} \text{competition_datasets} \\ &\bowtie_{\text{dataset_id}} \text{dataset} \\ &\bowtie_{\text{dataset_id}} \text{dataset_file} \end{aligned} \right) \right)$$


Рис. 6: План выполнения запроса 1.1

Таблица 21: Последовательность выполнения запроса 1.1

Шаг	Операция	Описание
1	Index Scan (user_username_key)	Поиск пользователя по индексу username, cost 0.31
2	Seq Scan (role)	Поиск роли по name, cost 1.52
3	Bitmap Heap Scan (team_member)	Участники команды по индексу user_id, cost 4.20
4	Nested Loop (participation)	Раскрытие участия команды по индексу, cost 0.03
5	Index Scan (competition_pkey)	Получение соревнований по первичному ключу, cost 0.02
6	Index Scan (competition_dataset_pkey)	Проверка датасетов соревнования, cost 0.02
7	Index Scan (dataset_pkey)	Фильтрация по имени датасета, cost 0.31
8	Seq Scan (dataset_file)	Чтение файлов датасета, cost 977.60
9	Hash Join (dataset_file)	Соединение файлов с датасетом, cost 14.32
10	Index Scan (submission)	Подтверждение наличия попыток, cost 0.48
11	Sort + Unique	Сортировка и устранение дубликатов, cost 0.01

comp_title	user_name	user_role	file_name
Forecast Open #722	iris5426	judge	labels.csv
Forecast Open #722	iris5426	judge	sample.tsv
Forecast Open #722	iris5426	judge	train.csv
(3 rows)			

Рис. 7: Результат запроса 1.1

Время выполнения запроса:

Planning Time: 9.725 ms

Execution Time: 15.272 ms

4.2 Запрос 1.2

Найти соревнования, в которых участвовал пользователь *A* в роли *B*, и команда которого отправила не менее трёх попыток с файлом датасета *C*.

```

1 SELECT DISTINCT
2     c.title      AS comp_title,
3     u.username   AS user_name,
4     r.name       AS user_role,
5     df.filename  AS file_name
6 FROM competition c
7 JOIN participation p

```

```

8      ON p.competition_id    = c.competition_id
9      JOIN team_member tm
10     ON tm.team_id          = p.team_id
11     JOIN "user" u
12     ON u.user_id           = tm.user_id
13     JOIN role r
14     ON r.role_id           = u.role_id
15     JOIN submission s
16     ON s.participation_id = p.participation_id
17     JOIN competition_dataset cd
18     ON cd.competition_id  = c.competition_id
19     JOIN dataset d
20     ON d.dataset_id       = cd.dataset_id
21     JOIN dataset_file df
22     ON df.dataset_id      = d.dataset_id
23 WHERE
24     u.username = $1
25     AND r.name = $2
26     AND d.name = $3
27 HAVING
28     COUNT(DISTINCT s.submission_id) >= 3
29 ORDER BY
30     c.title;

```

Реляционная формула запроса:

$$\begin{aligned}
 & \pi_{\text{title, username, role, filename}} \left(\right. \\
 & \quad \sigma_{\text{username}=A \wedge \text{role}=B \wedge \text{dataset}=C \wedge \text{COUNT}(\text{sub_id}) \geq 3} \left(\right. \\
 & \quad \quad \text{competition} \\
 & \quad \quad \bowtie_{\text{competition_id}} \text{participation} \\
 & \quad \quad \bowtie_{\text{team_id}} \text{team_member} \\
 & \quad \quad \bowtie_{\text{user_id}} \text{user} \\
 & \quad \quad \bowtie_{\text{role_id}} \text{role} \\
 & \quad \quad \bowtie_{\text{participation_id}} \text{submission} \\
 & \quad \quad \bowtie_{\text{competition_id}} \text{competition_dataset} \\
 & \quad \quad \bowtie_{\text{dataset_id}} \text{dataset} \\
 & \quad \quad \bowtie_{\text{dataset_id}} \text{dataset_file} \\
 & \quad \left. \right) \left. \right)
 \end{aligned}$$

Структура плана идентична запросу 1.1 с единственным отличием: перед финальным Sort и Unique планировщик вставляет GroupAggregate с фильтром $\text{COUNT}(\text{DISTINCT submission_id}) \geq 3$. Группировка выполняется после предварительной сортировки по (`competition_id`, `user_id`, `role_id`, `dataset_id`) (cost 0.02), GroupAggregate (cost 0.08) отсеивает группы с менее чем тремя уникальными попытками. Стоимостной профиль узлов нижних

уровней остаётся прежним: доминирует Seq Scan по `dataset_file` (cost 977.60), все остальные соединения дешевле на два-три порядка.

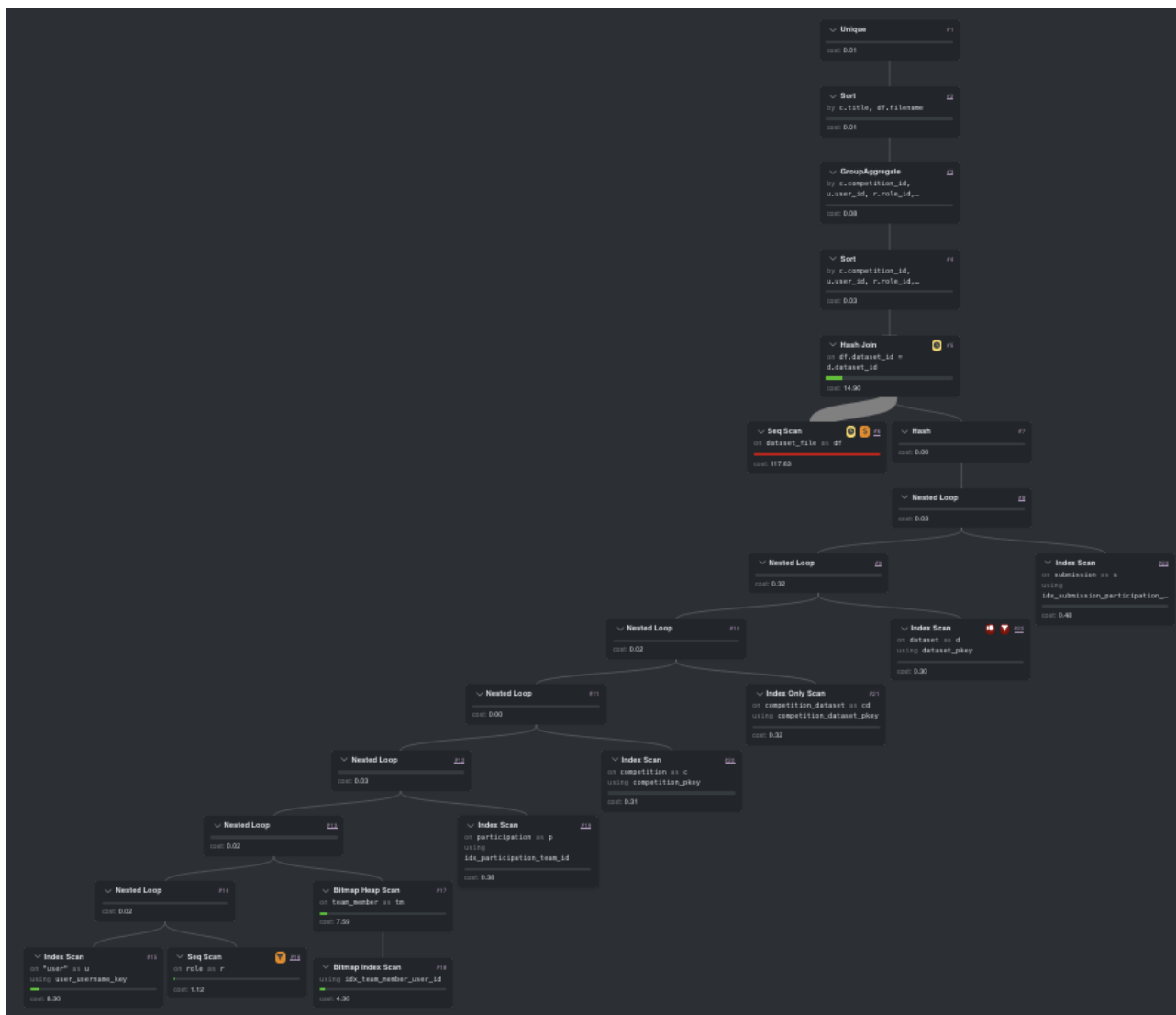


Рис. 8: План выполнения запроса 1.2

Таблица 22: Последовательность выполнения запроса 1.2

Шаг	Операция	Описание
1	Index Scan (user_username_key)	Поиск пользователя по индексу username, cost 0.31
2	Seq Scan (role)	Поиск роли по name, cost 1.52
3	Bitmap Heap Scan (team_member)	Участники команды по индексу user_id, cost 4.20
4	Nested Loop (participation)	Раскрытие участия команды по индексу, cost 0.03
5	Index Scan (competition_pkey)	Получение соревнований по первичному ключу, cost 0.02
6	Index Scan (competition_dataset_pkey)	Проверка датасетов соревнования, cost 0.02
7	Index Scan (dataset_pkey)	Фильтрация по имени датасета, cost 0.31
8	Seq Scan (dataset_file)	Чтение файлов датасета, cost 977.60
9	Hash Join (dataset_file)	Соединение файлов с датасетом, cost 14.32
10	Index Scan (submission)	Подтверждение наличия попыток, cost 0.48
11	Sort	Сортировка для группировки, cost 0.02
12	GroupAggregate	Фильтр $\text{COUNT}(\text{DISTINCT submission_id}) \geq 3$, cost 0.08
13	Sort + Unique	Сортировка и устранение дубликатов, cost 0.01

comp_title	user_name	user_role	file_name
Kernel Race #44	luna5807	organizer	notes.txt
Kernel Race #44	luna5807	organizer	test.csv
Kernel Race #44	luna5807	organizer	train.csv
(3 rows)			

Рис. 9: Результат запроса 1.2

Время выполнения запроса:

Planning Time: 9.725 ms

Execution Time: 15.272 ms

4.3 Запрос 2

Для метрики A , команды B и датасета C подсчитать число соревнований, в которых команда участвовала и использовала указанный датасет.

```

1 SELECT
2     m.name AS metric_name ,

```

```

3      t.name                                AS team_name ,
4      d.name                                AS dataset_name ,
5      COUNT(DISTINCT c.competition_id) AS competition_count
6 FROM team t
7      JOIN participation p
8          ON p.team_id                      = t.team_id
9      JOIN competition c
10         ON c.competition_id              = p.competition_id
11      JOIN configuration cfg
12         ON cfg.competition_id            = c.competition_id
13      JOIN metric m
14         ON m.metric_id                   = cfg.metric_id
15      JOIN competition_dataset cd
16         ON cd.competition_id             = c.competition_id
17      JOIN dataset d
18         ON d.dataset_id                  = cd.dataset_id
19 WHERE
20     m.name = $1
21     AND t.name = $2
22     AND d.name = $3
23 GROUP BY
24     m.name ,
25     t.name ,
26     d.name ;

```

Реляционная формула запроса:

$$\begin{aligned}
 & \pi_{\text{metric, team, dataset, competition_count}} \left(\right. \\
 & \quad \gamma_{\text{metric, team, dataset; COUNT(competition_id)}} \left(\right. \\
 & \quad \quad \sigma_{\text{metric}=A \wedge \text{team}=B \wedge \text{dataset}=C} \left(\right. \\
 & \quad \quad \quad \text{team} \\
 & \quad \quad \quad \bowtie_{\text{team_id}} \text{ participation} \\
 & \quad \quad \quad \bowtie_{\text{compe_id}} \text{ competition} \\
 & \quad \quad \quad \bowtie_{\text{competition_id}} \text{ configuration} \\
 & \quad \quad \quad \bowtie_{\text{metric_id}} \text{ metric} \\
 & \quad \quad \quad \bowtie_{\text{competition_id}} \text{ competition_dataset} \\
 & \quad \quad \quad \bowtie_{\text{dataset_id}} \text{ dataset} \\
 & \quad \quad \left. \right) \left. \right) \left. \right)
 \end{aligned}$$

Все соединения выполняются через Nested Loop по индексам, Seq Scan отсутствует. Запрос крайне дешёв: суммарный **cost** корневого узла (**GroupAggregate**) составляет около 0.25, поскольку планировщик сразу находит единственную команду через **team_name_key** и далее проходит цепочку индексов без сканирования больших таблиц.

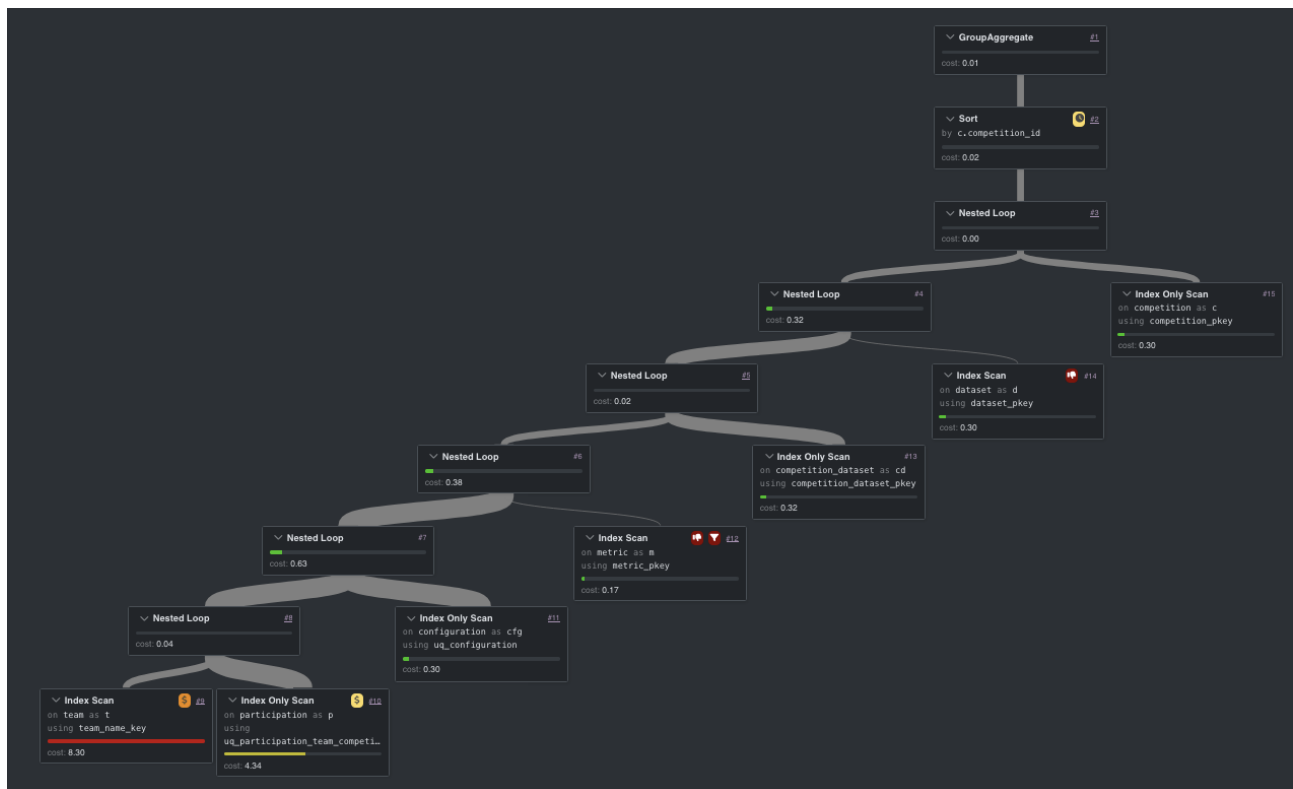


Рис. 10: План выполнения запроса 2

Таблица 23: Последовательность выполнения запроса 2

Шаг	Операция	Описание
1	Index Scan (team_name_key)	Поиск команды по индексу name
2	Index Only Scan (participation)	Участия команды по индексу
3	Index Only Scan (configuration)	Конфигурация соревнования по индексу
4	Index Scan (metric_pkey)	Фильтрация метрики по имени
5	Index Only Scan (competition_dataset)	Проверка датасета соревнования
6	Index Only Scan (competition_pkey)	Получение соревнования по ключу
7	Index Scan (dataset_pkey)	Фильтрация датасета по имени
8	Sort	Сортировка по ключу группировки
9	GroupAggregate	Подсчёт уникальных соревнований

metric_name	team_name	dataset_name	competition_count
accuracy	Silent Monkeys 819559	vision_38	1
(1 row)			

Рис. 11: Результат запроса 2

Время выполнения запроса:

Planning Time: 9.977 ms

Execution Time: 1.248 ms

4.4 Запрос 3

Для каждой из первых 20 команд подсчитать число уникальных соревнований и число попыток.

```
1 SELECT
2     t.name                               AS team_name ,
3     COUNT(DISTINCT p.competition_id) AS competition_count ,
4     COUNT(DISTINCT s.submission_id) AS submission_count
5 FROM (
6     SELECT team_id, name
7     FROM team
8     ORDER BY team_id
9     LIMIT 20
10 ) t
11 JOIN participation p ON p.team_id = t.team_id
12 JOIN submission s ON s.participation_id =
13     p.participation_id
14 GROUP BY
15     t.team_id,
16     t.name
17 ORDER BY
18     t.team_id;
```

Реляционная формула запроса:

$$\begin{aligned} & \pi_{\text{name, competition_count, submission_count}} \left(\right. \\ & \quad \gamma_{\text{team_id; COUNT(competition_id) COUNT(submission_id)}} \left(\right. \\ & \quad \quad \sigma_{\text{rank} \leq 20} \left(\right. \\ & \quad \quad \quad \text{team} \\ & \quad \quad \quad \bowtie_{\text{team_id}} \text{participation} \\ & \quad \quad \quad \bowtie_{\text{participation_id}} \text{submission} \\ & \quad \quad \left. \right) \\ & \quad \left. \right) \end{aligned}$$

Планировщик берёт 20 команд через `Limit` по `team_pkey` (cost 682.76). Участия и попытки раскрываются двумя `Nested Loop` по индексам `idx_participation` (cost 8.14) и `idx_submission_participation_id` (cost 0.48). Перед агрегацией выполняется `Incremental Sort` (cost 5.94), поскольку данные уже частично упорядочены по `team_id`. Корневой `GroupAggregate` (cost 1.52) считает `COUNT(DISTINCT)` по каждой из 20 групп.

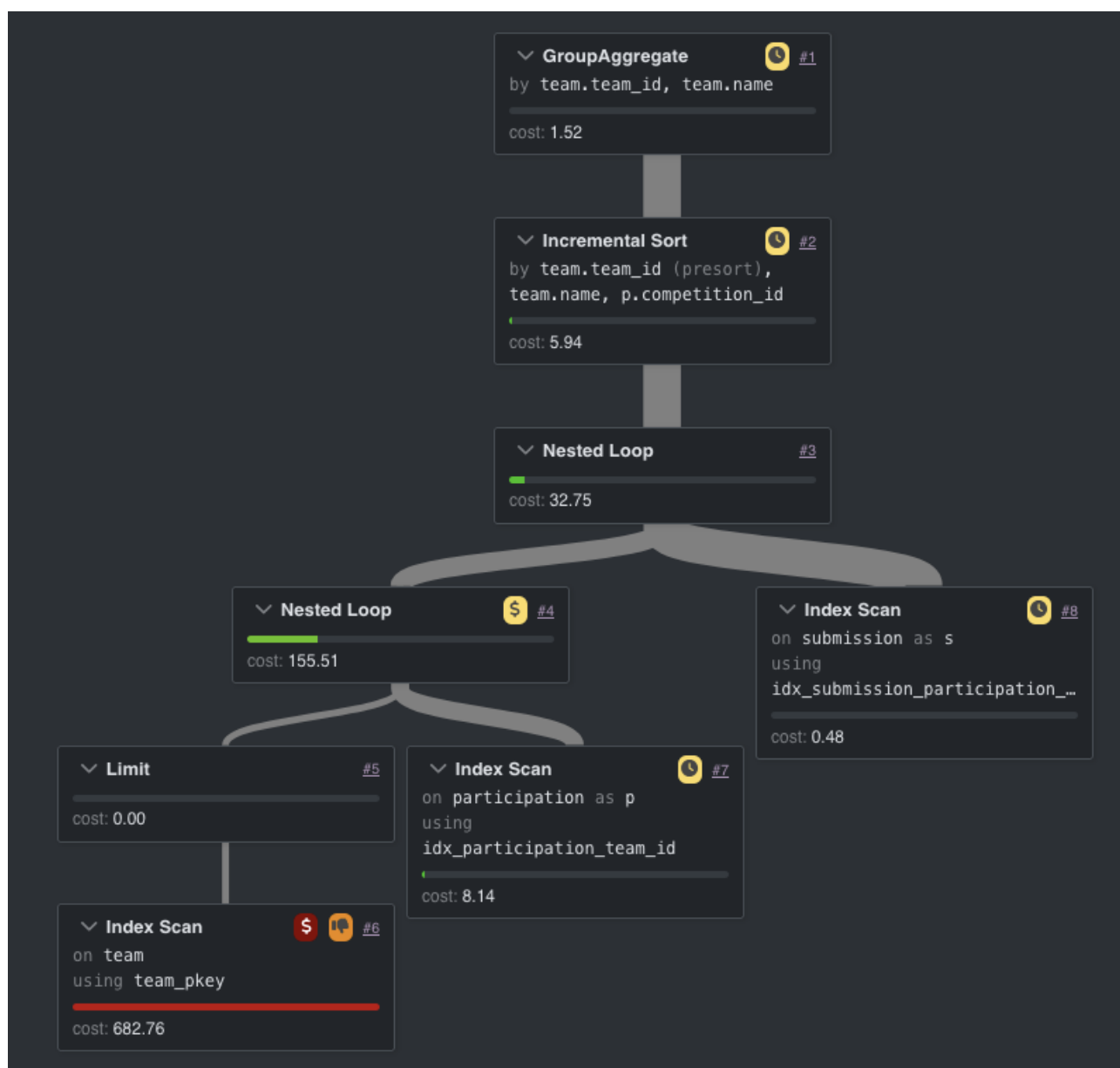


Рис. 12: План выполнения запроса 3

Таблица 24: Последовательность выполнения запроса 3

Шаг	Операция	Описание
1	Index Scan + Limit (team_pkey)	Первые 20 команд по первичному ключу, cost 682.76
2	Nested Loop (idx_participation_team_id)	Участия каждой команды по индексу, cost 8.14
3	Nested Loop (idx_submission_participation_id)	Попытки каждого участия по индексу, cost 0.48
4	Incremental Sort	Сортировка по ключам группировки, cost 5.94
5	GroupAggregate	Подсчёт уникальных соревнований и попыток, cost 1.52

team_name	competition_count	submission_count
Spark Otters 75697	2	5
Kernel Otters 748875	4	5
Silent Wizards 596339	1	2
Vector Otters 24366	3	7
Spark Otters 190166	4	9
Bayes Falcons 747636	2	4
Quantum Otters 99678	1	2
Kernel Giants 875204	1	2
Spark Monkeys 743267	3	6
Gradient Giants 189258	2	4
Quantum Nomads 497509	5	13
Gradient Pirates 118800	5	9
Vector Otters 785250	3	8
Gradient Giants 907809	3	5
Silent Otters 797456	2	3
Boosted Nomads 900031	3	5
Spark Giants 614004	2	6
Bayes Monkeys 591440	1	1
Gradient Wizards 393185	2	3
Quantum Racers 796556	4	6
(20 rows)		

Рис. 13: Результат запроса 3

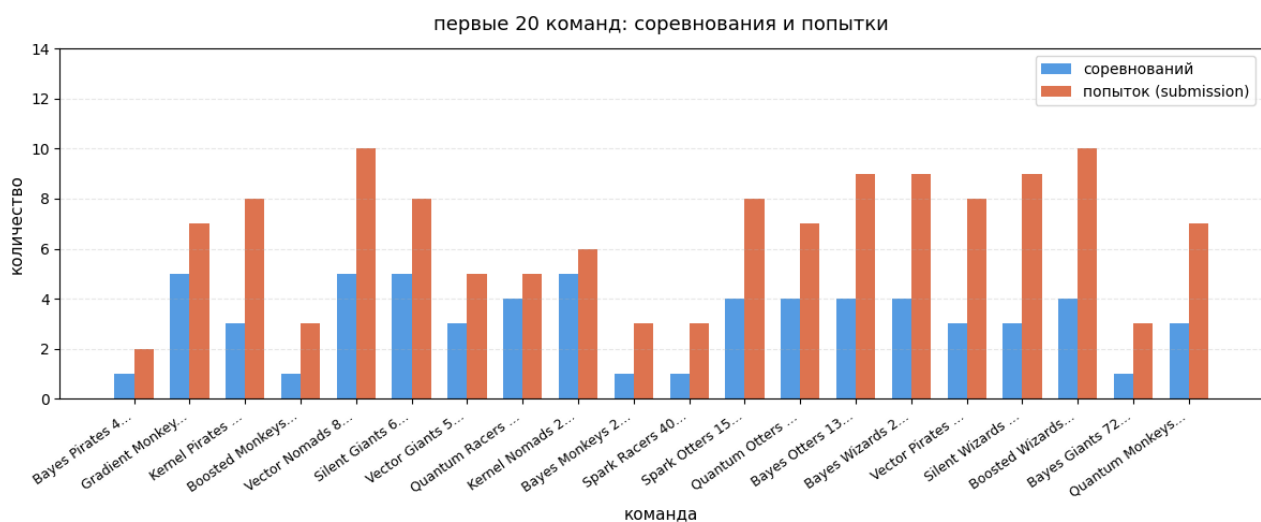


Рис. 14: Гистограмма числа соревнований и попыток для первых 20 команд

Время выполнения запроса:

Planning Time: 3.927 ms

Execution Time: 1.677 ms

4.5 Запрос 4

Найти пользователей с максимальным числом участия в соревнованиях.

Вариант 4.1 — подзапрос с MAX

```
1 SELECT
2     u.user_id,
3     u.username,
4     COUNT(p.participation_id) AS p_count
5 FROM "user" u
6     JOIN team_member tm ON tm.user_id = u.user_id
7     JOIN participation p ON p.team_id = tm.team_id
8 GROUP BY
9     u.user_id,
10    u.username
11 HAVING COUNT(p.participation_id) = (
12     SELECT MAX(p2_count)
13     FROM (
14         SELECT COUNT(p2.participation_id) AS p2_count
15         FROM team_member tm2
16             JOIN participation p2 ON p2.team_id = tm2.team_id
17         GROUP BY tm2.user_id
18     ) counts
19 );
```

Вариант 4.2 — подзапрос с >= ALL

```
1 SELECT
2     u.user_id,
3     u.username,
4     COUNT(p.participation_id) AS p_count
5 FROM "user" u
6     JOIN team_member tm ON tm.user_id = u.user_id
7     JOIN participation p ON p.team_id = tm.team_id
8 GROUP BY
9     u.user_id,
10    u.username
11 HAVING COUNT(p.participation_id) >= ALL (
12     SELECT COUNT(p2.participation_id)
13     FROM team_member tm2
14         JOIN participation p2 ON p2.team_id = tm2.team_id
15     GROUP BY tm2.user_id
16 )
17 ORDER BY p_count DESC;
```

Реляционная формула запроса (общая для обоих вариантов):

$$\pi_{\text{user_id, username, p_count}} \left(\sigma_{\text{p_count}=\text{MAX}(\text{p_count})} \left(\gamma_{\text{user_id}; \text{COUNT}(\text{participation_id})} \left(\begin{aligned} &\text{user} \\ &\bowtie_{\text{user_id}} \text{team_member} \\ &\bowtie_{\text{team_id}} \text{participation} \end{aligned} \right) \right) \right)$$

Вариант 4.1. Подзапрос вычисляется однократно как `InitPlan: Hash Join` соединяет `team_member` и `participation` по `team_id`, `HashAggregate` группирует по `user_id`, `Aggregate` берёт `MAX`. Результат передаётся как константа `$0`. Основной запрос выполняет два `Hash Join` и `HashAggregate` с фильтром `count = $0`. Из 32 350 групп условию удовлетворяют 3.

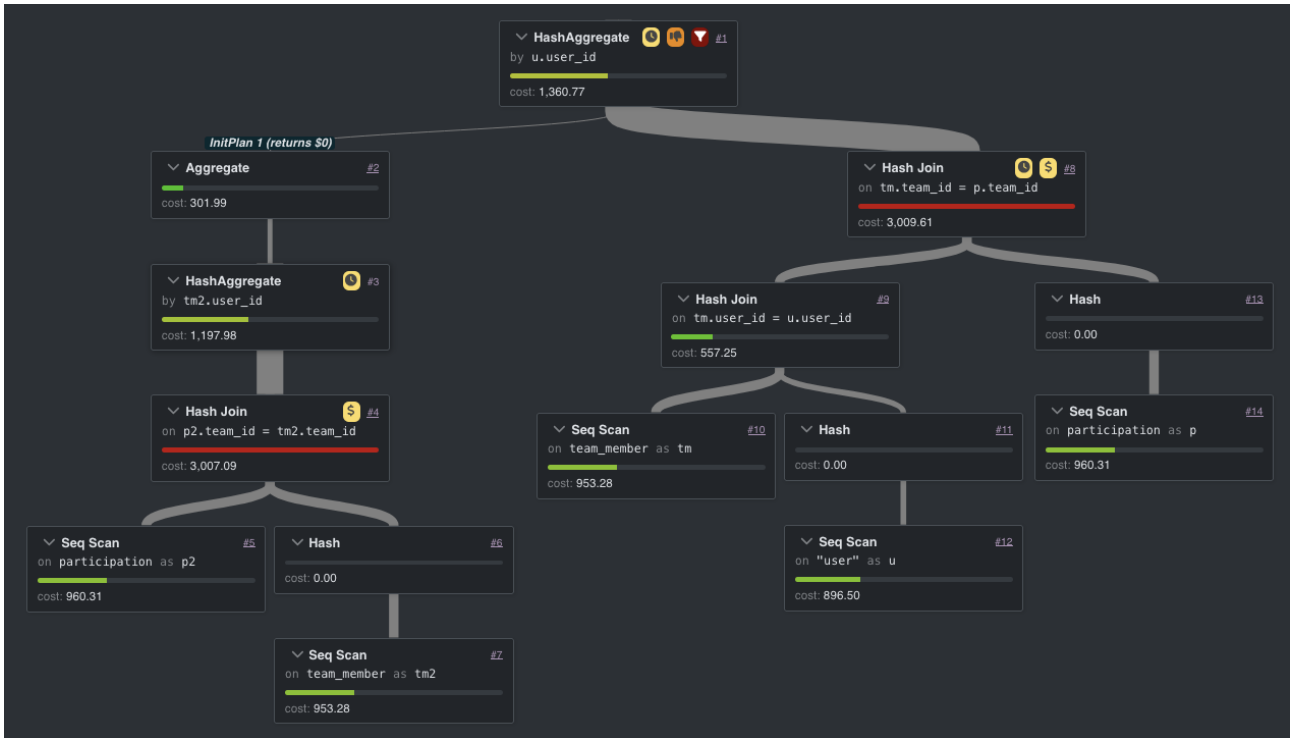


Рис. 15: План выполнения запроса 4.1

Таблица 25: Последовательность выполнения запроса 4.1

Шаг	Операция	Описание
1	Seq Scan (team_member tm2)	Чтение участников для подзапроса
2	Seq Scan (participation p2)	Чтение участия для подзапроса
3	Hash Join	Соединение tm2 и p2 по team_id
4	HashAggregate	Группировка по user_id, подсчёт участия
5	Aggregate	Вычисление MAX по счётчикам
6	Seq Scan (user)	Чтение таблицы пользователей
7	Seq Scan (team_member tm)	Чтение участников команд
8	Hash Join (team_member)	Соединение пользователей с участниками по user_id
9	Seq Scan (participation p)	Чтение участия
10	Hash Join (participation)	Соединение с участиями по team_id
11	HashAggregate + Filter	Группировка и фильтр по максимуму

Вариант 4.2. Конструкция `>= ALL` порождает SubPlan, выполняемый для каждой из 32 350 групп. Планировщик материализует результат подзапроса, однако сравнение повторяется на каждой итерации. Основной запрос использует Merge Join и Nested Loop с Memoize. Из-за сложности плана активируется JIT-компиляция (суммарно ~109 ms накладных расходов).

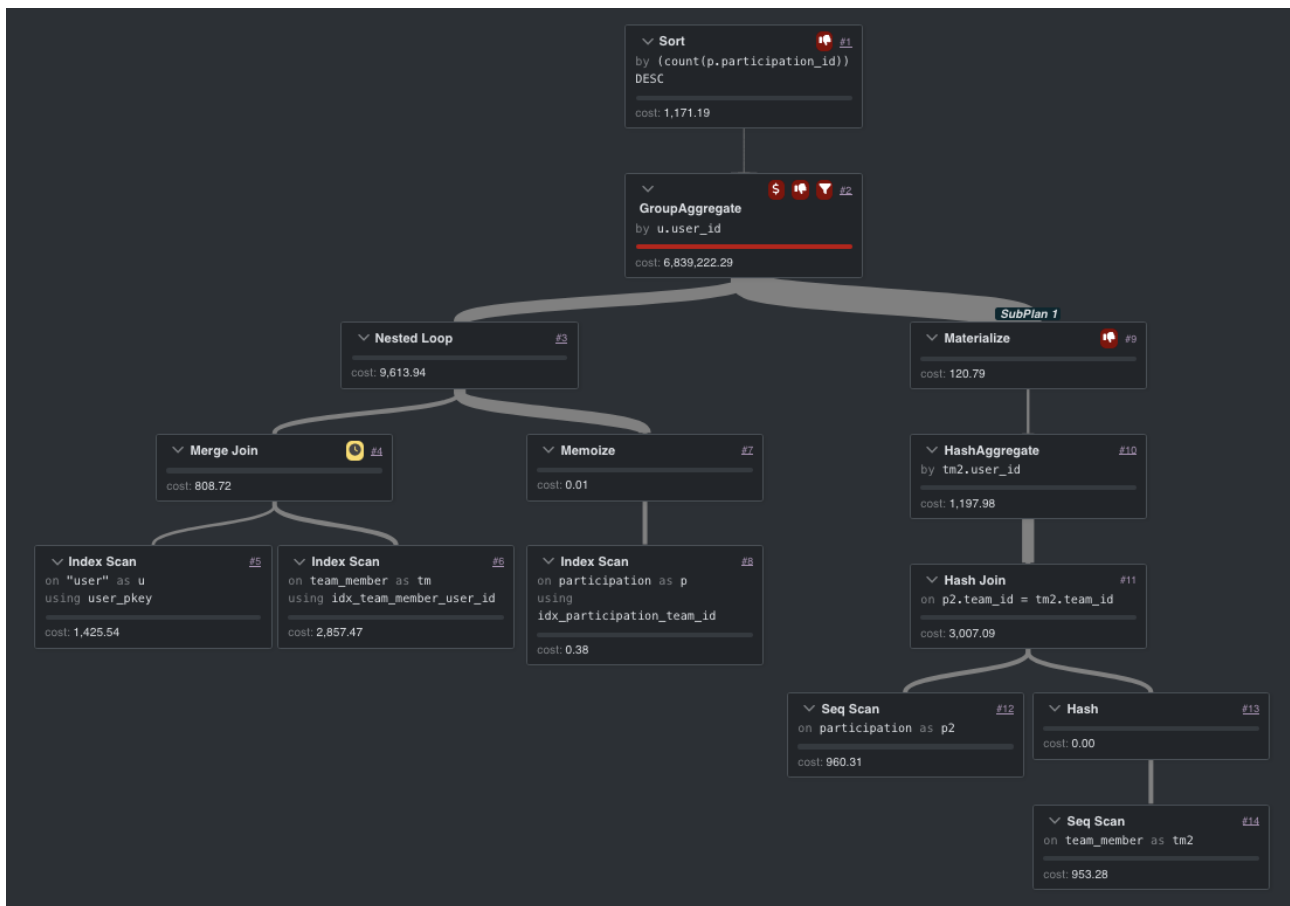


Рис. 16: План выполнения запроса 4.2

Таблица 26: Последовательность выполнения запроса 4.2

Шаг	Операция	Описание
1	Seq Scan (<code>team_member</code> tm2)	Чтение участников для подзапроса
2	Seq Scan (<code>participation</code> p2)	Чтение частей для подзапроса
3	Hash Join	Соединение tm2 и p2 по <code>team_id</code>
4	HashAggregate	Группировка по <code>user_id</code> в подзапросе
5	Materialize	Материализация результата подзапроса
6	Index Scan (<code>user_pkey</code>)	Чтение пользователей по первичному ключу
7	Seq Scan (<code>team_member</code> tm)	Чтение участников команд
8	Sort (<code>team_member</code>)	Сортировка по <code>user_id</code> для Merge Join
9	Merge Join	Соединение пользователей с участниками по <code>user_id</code>
10	Memoize + Index Scan (<code>participation</code>)	Участия по индексу с кэшированием
11	GroupAggregate + SubPlan Filter	Группировка и сравнение с подзапросом
12	Sort	Сортировка по <code>p_count</code> DESC

Время выполнения запроса 4.1:

Planning Time: 2.289 ms

Execution Time: 113.487 ms

Время выполнения запроса 4.2:

Planning Time: —

JIT Total: 109.038 ms

Execution Time: 238.436 ms

Сравнение вариантов. Оба запроса возвращают 3 пользователя с максимальным числом частей. В варианте 4.1 подзапрос вычисляется однократно как `InitPlan` и его результат используется как константа `$0`. В варианте 4.2 `>= ALL` порождает `SubPlan`, обращающийся к материализованному результату на каждой итерации, плюс JIT добавляет ~109 ms накладных расходов. Вариант 4.1 быстрее примерно в 2 раза (113 ms против 238 ms) и является предпочтительным.

<code>user_id</code>	<code>username</code>	<code>p_count</code>
2268	pixel5479	27
16689	vector6906	27
23295	nova2144	27
(3 rows)		

Рис. 17: Результат запроса 4

4.6 Запрос 5

Подсчитать число пользователей с одинаковым количеством участия в командах.

```
1 SELECT
2     tm_count ,
3     COUNT(user_id) AS user_count
4 FROM (
5     SELECT
6         u.user_id ,
7         COUNT(tm.member_id) AS tm_count
8     FROM "user" u
9     LEFT JOIN team_member tm ON tm.user_id = u.user_id
10    GROUP BY u.user_id
11 ) counts
12 GROUP BY
13     tm_count
14 ORDER BY
15     tm_count DESC;
```

Реляционная формула запроса:

$$\pi_{tm_count, user_count} \left(\gamma_{tm_count; COUNT(user_id)} \left(\gamma_{user_id; COUNT(member_id)} \left(user \bowtie (left)_{user_id} team_member \right) \right) \right)$$

Внутренний подзапрос считает число участия каждого пользователя через `LEFT JOIN` с `team_member` — пользователи без участия получают `tm_count = 0`. Планировщик реализует это через `Hash Right Join`: 32350 пользователей загружаются в хеш-таблицу через `Index Only Scan` по `user_pkey` (cost 849.54), затем `Seq Scan` по `team_member` (cost 953.28) соединяется с ней. Внутренний `HashAggregate` группирует по `user_id` (cost 614.64), внешний — по значению `tm_count` (cost 487.25).

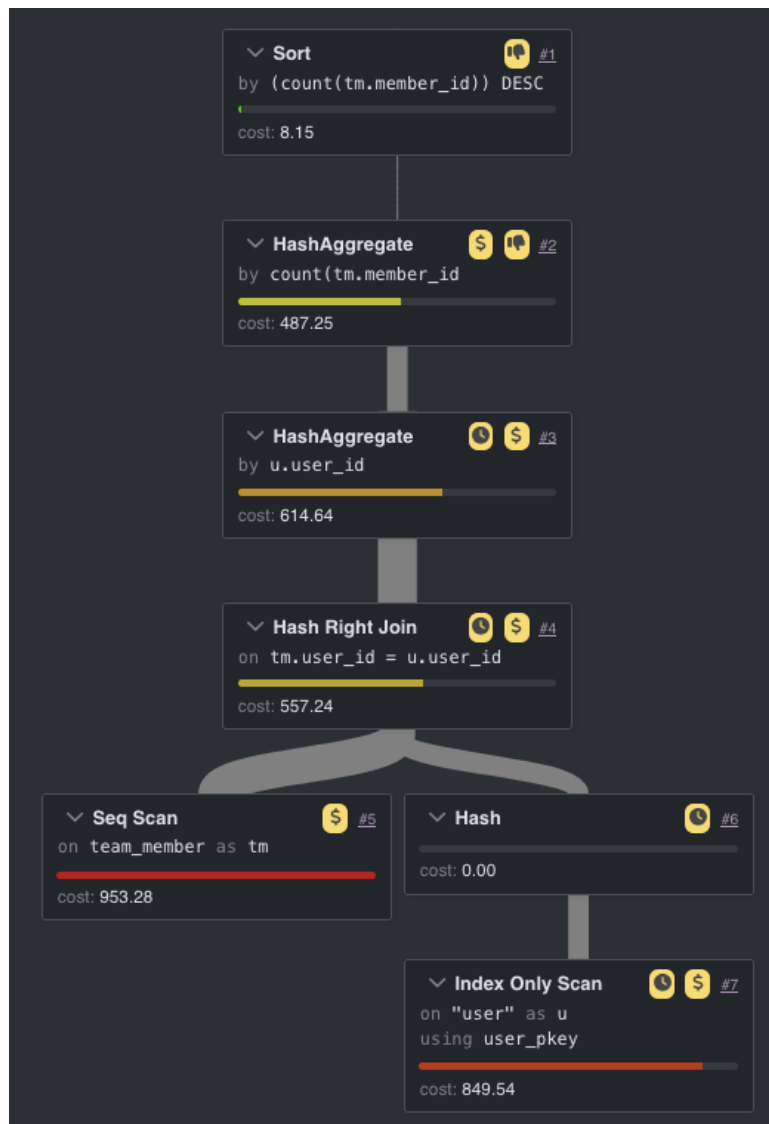


Рис. 18: План выполнения запроса 5

Таблица 27: Последовательность выполнения запроса 5

Шаг	Операция	Описание
1	Index Only Scan (user_pkey)	Загрузка всех пользователей в хеш-таблицу, cost 849.54
2	Seq Scan (team_member)	Полный скан участников команд, cost 953.28
3	Hash Right Join	Соединение участников с пользователями по user_id, cost 557.24
4	HashAggregate	Подсчёт tm_count для каждого пользователя, cost 614.64
5	HashAggregate	Группировка по значению tm_count, cost 487.25
6	Sort	Сортировка по tm_count по убыванию, cost 8.15

```
tm_count DESC;
tm_count | user_count
-----+-----
      8 |         2
      7 |        12
      6 |        62
      5 |       284
      4 |      892
      3 |     2410
      2 |     5138
      1 |     7033
      0 |     4716
(9 rows)
```

Рис. 19: Результат запроса 5

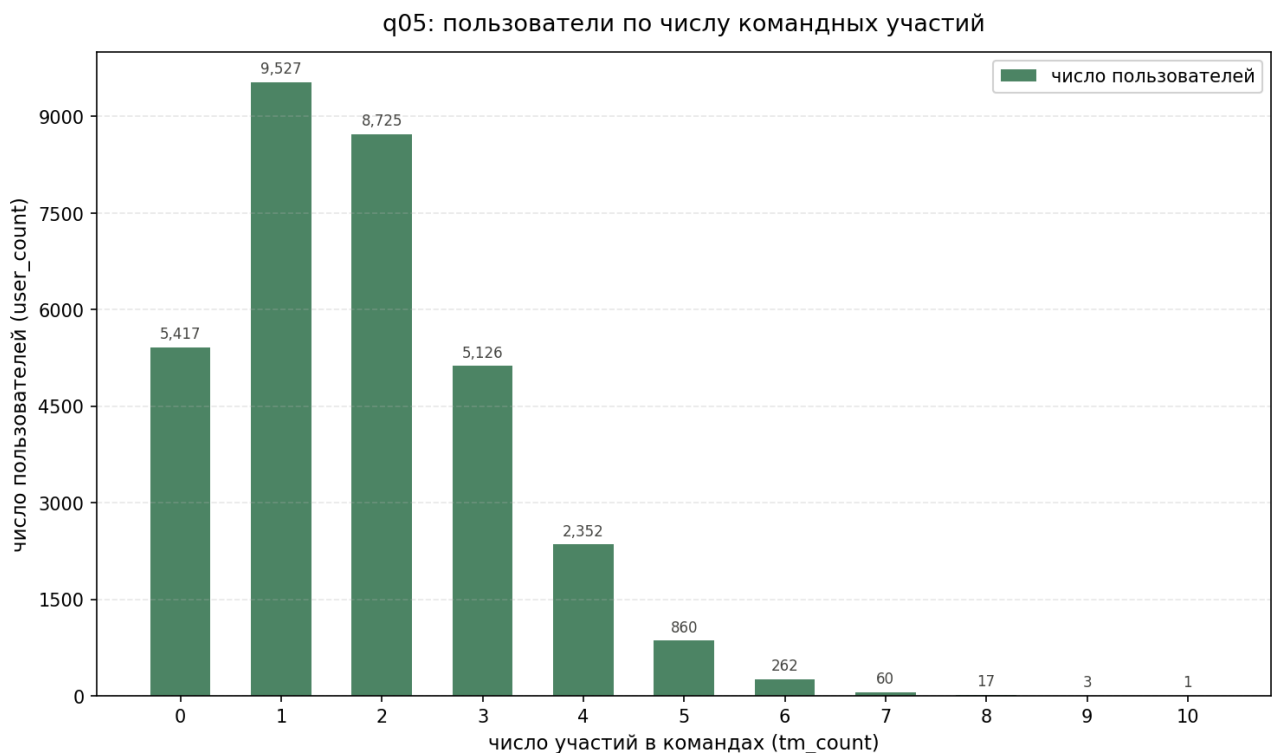


Рис. 20: Распределение пользователей по числу командных участия

Время выполнения запроса:

Planning Time: 1.802 ms

Execution Time: 48.975 ms

4.7 Запрос 6

Найти команды, которые участвовали в соревнованиях чаще, чем команда А.

Вариант 6.1 — подзапрос в FROM

```
1 SELECT
2     ref.team_name           AS team_a,
3     ref.participation_count AS score_a,
4     t.name                  AS team_b,
5     COUNT(p.participation_id) AS score_b
6 FROM team t
7     JOIN participation p ON p.team_id = t.team_id,
8     (
9         SELECT
10             t2.name           AS team_name,
11             COUNT(p2.participation_id) AS participation_count
12         FROM team t2
13             JOIN participation p2 ON p2.team_id = t2.team_id
14         WHERE
15             t2.name = 'Quantum Falcons 470319'
16         GROUP BY
17             t2.team_id,
18             t2.name
19     ) ref
20 GROUP BY
21     t.team_id,
22     t.name,
23     ref.team_name,
24     ref.participation_count
25 HAVING COUNT(p.participation_id) > ref.participation_count
26 ORDER BY
27     score_b DESC;
```

Вариант 6.2 — двойной InitPlan

```
1 SELECT
2     'Quantum Falcons 470319' AS team_a,
3     (
4         SELECT COUNT(p2.participation_id)
5         FROM team t2
6             JOIN participation p2 ON p2.team_id = t2.team_id
7         WHERE
8             t2.name = 'Quantum Falcons 470319'
9         GROUP BY
10             t2.team_id
11     ) AS score_a,
12     t.name AS team_b,
13     COUNT(p.participation_id) AS score_b
14 FROM team t
15     JOIN participation p ON p.team_id = t.team_id
16 GROUP BY
17     t.team_id,
18     t.name
19 HAVING COUNT(p.participation_id) > (
```

```

20 SELECT COUNT(p2.participation_id)
21 FROM team t2
22      JOIN participation p2 ON p2.team_id = t2.team_id
23 WHERE
24      t2.name = 'Quantum Falcons 470319'
25 GROUP BY
26      t2.team_id
27 )
28 ORDER BY
29      score_b DESC;

```

Реляционная формула запроса (общая для обоих вариантов):

$$\begin{aligned}
 & \pi_{\text{team_a}, \text{score_a}, \text{team_b}, \text{score_b}} \left(\right. \\
 & \quad \sigma_{\text{score_b} > \text{score_a}} \left(\right. \\
 & \quad \quad \gamma_{\text{team_id}; \text{COUNT}(\text{participation_id}) \rightarrow \text{score_b}} \left(\right. \\
 & \quad \quad \quad \text{team} \\
 & \quad \quad \quad \bowtie_{\text{team_id}} \text{participation} \\
 & \quad \quad \left. \right) \\
 & \quad \times \gamma_{\text{team_id}; \text{COUNT}(\text{participation_id}) \rightarrow \text{score_a}} \left(\right. \\
 & \quad \quad \sigma_{\text{name}=A}(\text{team}) \\
 & \quad \quad \bowtie_{\text{team_id}} \text{participation} \\
 & \quad \left. \right) \left. \right)
 \end{aligned}$$

Вариант 6.1. Подзапрос в **FROM** вычисляется однократно как **GroupAggregate** и материализуется в одну строку. Строка **ref** присоединяется декартовым произведением ко всем строкам основного запроса, что вынуждает планировщик включать **ref.team_name** и **ref.participation_count** в ключи **GROUP BY** — хеш-таблица агрегации расширяется до 2193 кБ.

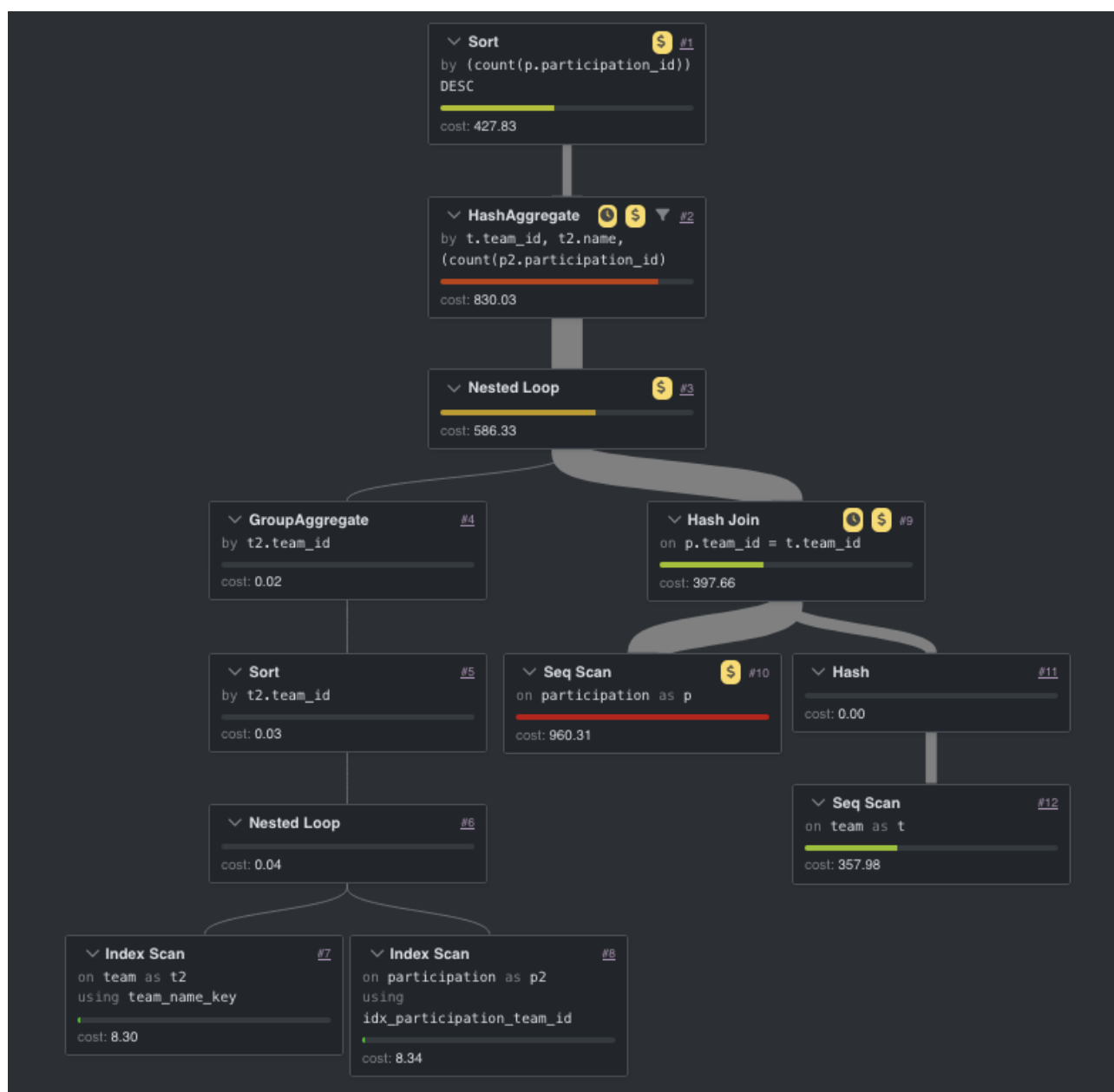


Рис. 21: План выполнения запроса 6.1

Таблица 28: Последовательность выполнения запроса 6.1

Шаг	Операция	Описание
1	Index Scan (team_name_key)	Поиск команды A по имени
2	Bitmap Heap Scan (participation p2)	Участия команды A по индексу
3	GroupAggregate	Подсчёт числа участий команды A
4	Seq Scan (participation p)	Полное чтение всех участий
5	Seq Scan (team t)	Полное чтение всех команд
6	Hash Join	Соединение участий с командами по team_id
7	Nested Loop	Декартово произведение с ref
8	HashAggregate + Filter	Группировка и фильтр HAVING
9	Sort	Сортировка по score_b DESC

Время выполнения запроса 6.1:

Planning Time: 1.238 ms

Execution Time: 47.329 ms

Вариант 6.2. Оба вхождения подзапроса выносятся в InitPlan 1 (для `score_a` в `SELECT`) и InitPlan 2 (для `HAVING`), каждый вычисляется однократно через `Index Scan` + `Bitmap Heap Scan` + `GroupAggregate`. Результаты сохраняются как константы \$1 и \$3. Основной запрос группирует только по `team_id` — хеш-таблица агрегации компактнее (1425 кБ).

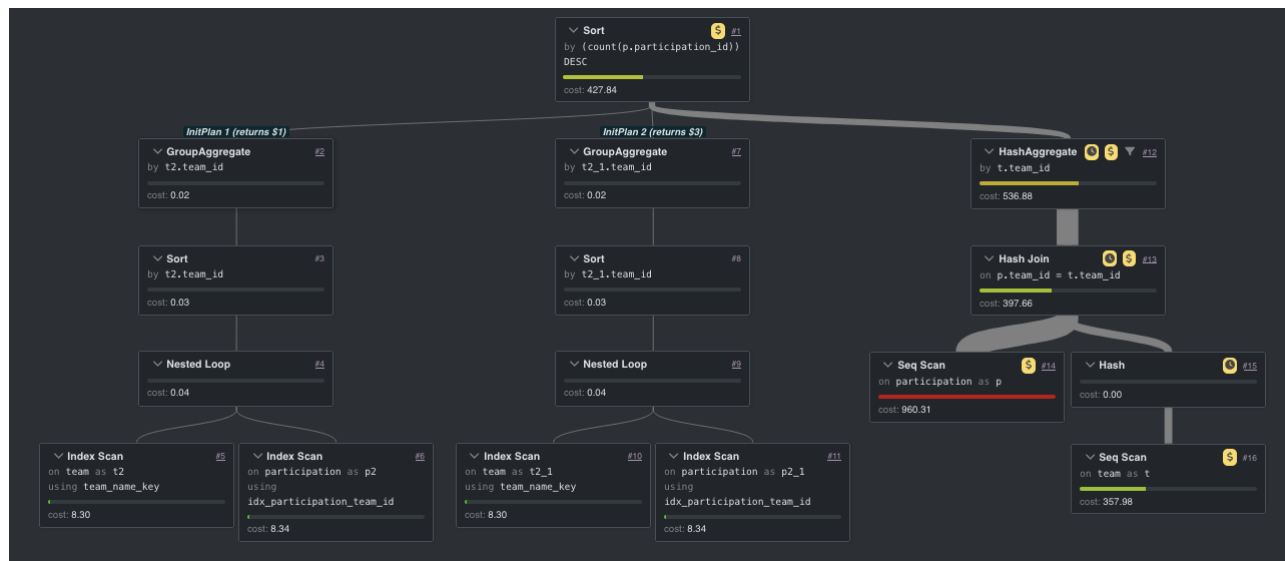


Рис. 22: План выполнения запроса 6.2

Таблица 29: Последовательность выполнения запроса 6.2

Шаг	Операция	Описание
1	Index Scan (<code>team_name_key</code>)	Поиск команды <i>A</i> (InitPlan 1)
2	Bitmap Heap Scan (<code>participation p2</code>)	Участия команды <i>A</i> (InitPlan 1)
3	GroupAggregate	Подсчёт участия → \$1 (InitPlan 1)
4	Index Scan (<code>team_name_key</code>)	Поиск команды <i>A</i> (InitPlan 2)
5	Bitmap Heap Scan (<code>participation p2</code>)	Участия команды <i>A</i> (InitPlan 2)
6	GroupAggregate	Подсчёт участия → \$3 (InitPlan 2)
7	Seq Scan (<code>participation p</code>)	Полное чтение всех участия
8	Seq Scan (<code>team t</code>)	Полное чтение всех команд
9	Hash Join	Соединение участия с командами по <code>team_id</code>
10	HashAggregate + Filter	Группировка и фильтр <code>count > \$3</code>
11	Sort	Сортировка по <code>score_b DESC</code>

Время выполнения запроса 6.2:

Planning Time: 3.418 ms

Execution Time: 45.991 ms

Сравнение вариантов. Оба запроса возвращают 8049 команд. Разница во времени незначительна (47.3 ms против 46.0 ms, ~3%), однако вариант 6.2 предпочтительнее структурно: компактные ключи группировки сокращают хеш-таблицу агрегации на 35% (1425 kB против 2193 kB), а `InitPlan` с константным результатом эффективнее декартова произведения с агрегатом в ключах `GROUP BY`.

team_a	score_a	team_b	score_b
Quantum Falcons 470319	1	Silent Racers 971654	5
Quantum Falcons 470319	1	Quantum Wizards 650608	5
Quantum Falcons 470319	1	Quantum Giants 253377	5
Quantum Falcons 470319	1	Gradient Giants 12414	5
Quantum Falcons 470319	1	Kernel Falcons 4627	5
Quantum Falcons 470319	1	Quantum Wizards 350224	5
Quantum Falcons 470319	1	Bayes Pirates 425237	5
Quantum Falcons 470319	1	Gradient Pirates 254747	5

Рис. 23: Результат запроса 6

4.8 Запрос 7

Найти пользователей, которые ни разу не участвовали в попытке решения, использующей файл датасета *A*.

```

1 SELECT
2     u.username AS user_name
3 FROM "user" u
4 WHERE u.user_id NOT IN (
5     SELECT DISTINCT tm.user_id
6     FROM participation p2
7         JOIN team_member tm
8             ON tm.team_id = p2.team_id
9         JOIN submission s
10            ON s.participation_id = p2.participation_id
11        JOIN competition_dataset cd
12            ON cd.competition_id = p2.competition_id
13        JOIN dataset_file fd
14            ON fd.dataset_id = cd.dataset_id
15        WHERE fd.filename = $1
16 )
17 GROUP BY
18     u.user_id,
19     u.username;
```

Реляционная формула запроса:

$$\begin{aligned}
 &\pi_{\text{username}} \left(\right. \\
 &\quad \text{user} \text{ --- } \pi_{\text{user_id}} \left(\right. \\
 &\quad \quad \sigma_{\text{filename}=A} \left(\right. \\
 &\quad \quad \text{participation} \\
 &\quad \quad \bowtie_{\text{team_id}} \text{team_member} \\
 &\quad \quad \bowtie_{\text{participation_id}} \text{submission} \\
 &\quad \quad \bowtie_{\text{competition_id}} \text{competition_dataset} \\
 &\quad \quad \bowtie_{\text{dataset_id}} \text{dataset_file} \\
 &\quad \left. \right) \\
 &\left. \right)
 \end{aligned}$$

Подзапрос собирает **user_id** пользователей, чья команда подавала попытку в соревновании с файлом *A*. Планировщик строит хеш-таблицу через цепочку **Hash Join**: фильтрует **dataset_file** по имени (cost 127.54), затем последовательно присоединяет **competition_dataset** (cost 122.43), **participation** и **submission** (cost 1040.15), **team_member** (cost 3419.13); результат агрегируется в **HashAggregate** (cost 806.22). Внешний **Seq Scan** по **user** (cost 1037.77) проверяет каждую из 32 350 строк через **NOT IN** по хеш-таблице за $O(1)$.



Рис. 24: План выполнения запроса 7

Таблица 30: Последовательность выполнения запроса 7

Шаг	Операция	Описание
1	Seq Scan (dataset_file)	Фильтрация файлов по имени, cost 127.54
2	Hash Join (competition_dataset)	Соединение с датасетами по dataset_id, cost 122.43
3	Hash Join (participation)	Соединение с участиями по competition_id, cost 1040.15
4	Index Only Scan (submission)	Получение попыток по индексу participation_id
5	Hash Join (team_member)	Соединение с участниками по team_id, cost 3419.13
6	HashAggregate	Сбор уникальных user_id, cost 806.22
7	Seq Scan (user)	Фильтрация пользователей через NOT IN, cost 1037.77
8	HashAggregate	Группировка итогового результата, cost 202.19

```

user_name
-----
luna8745
comet4257
nova4849
iris6070

```

Рис. 25: Результат запроса 7

Время выполнения запроса:

Planning Time: 5.823 ms

Execution Time: 106.522 ms

4.9 Запрос 8

Для каждой из первых 15 команд и каждой роли подсчитать число участников, включая комбинации с нулевым числом участников.

```

1 SELECT
2     t.name                AS team_name ,
3     tr.name               AS team_role ,
4     COUNT(DISTINCT tm.user_id) AS member_count
5 FROM (
6     SELECT team_id, name
7     FROM team
8     ORDER BY name
9     LIMIT 15
10 ) t

```

```

11 CROSS JOIN team_role tr
12 LEFT JOIN team_member tm
13     ON  tm.team_id      = t.team_id
14     AND tm.team_role_id = tr.team_role_id
15 GROUP BY
16     t.team_id,
17     t.name,
18     tr.team_role_id,
19     tr.name
20 ORDER BY
21     t.name,
22     tr.name;

```

Реляционная формула запроса:

$$\begin{aligned}
 & \pi_{\text{team, role, member_count}} \left(\right. \\
 & \quad \gamma_{\text{team, role; COUNT(user_id)}} \left(\right. \\
 & \quad \quad \sigma_{\text{rank} \leq 15}(\text{team}) \\
 & \quad \quad \times \text{team_role} \\
 & \quad \quad \text{team_id, team_role_id team_member} \\
 & \left. \right) \left. \right)
 \end{aligned}$$

CROSS JOIN формирует все комбинации команда × роль, включая те, где участников нет. LEFT JOIN с team_member подтягивает реальных участников; остальные комбинации получают COUNT = 0. Планировщик реализует CROSS JOIN через Nested Loop между Seq Scan по team_role (cost 17.20) и материализованным подзапросом первых 15 команд по team_name_key (cost 1472.73). LEFT JOIN реализован через Nested Loop Left Join с Memoize по индексу idx_team_member_team_id (cost 8.34). Финальный GroupAggregate (cost 216.00) считает COUNT(DISTINCT user_id).

Таблица 31: Последовательность выполнения запроса 8

Шаг	Операция	Описание
1	Index Scan + Limit (team_name_key)	Первые 15 команд по индексу name, cost 1472.73
2	Seq Scan (team_role)	Все роли команды, cost 17.20
3	Nested Loop (Materialize)	Декартово произведение команд и ролей
4	Memoize + Index Scan (team_member)	Участники по team_id и team_role_id, cost 8.34
5	Nested Loop Left Join	Соединение комбинаций с участниками
6	Incremental Sort	Сортировка по ключам группировки
7	GroupAggregate	Подсчёт уникальных участников, cost 216.00
8	Incremental Sort	Финальная сортировка по имени команды и роли

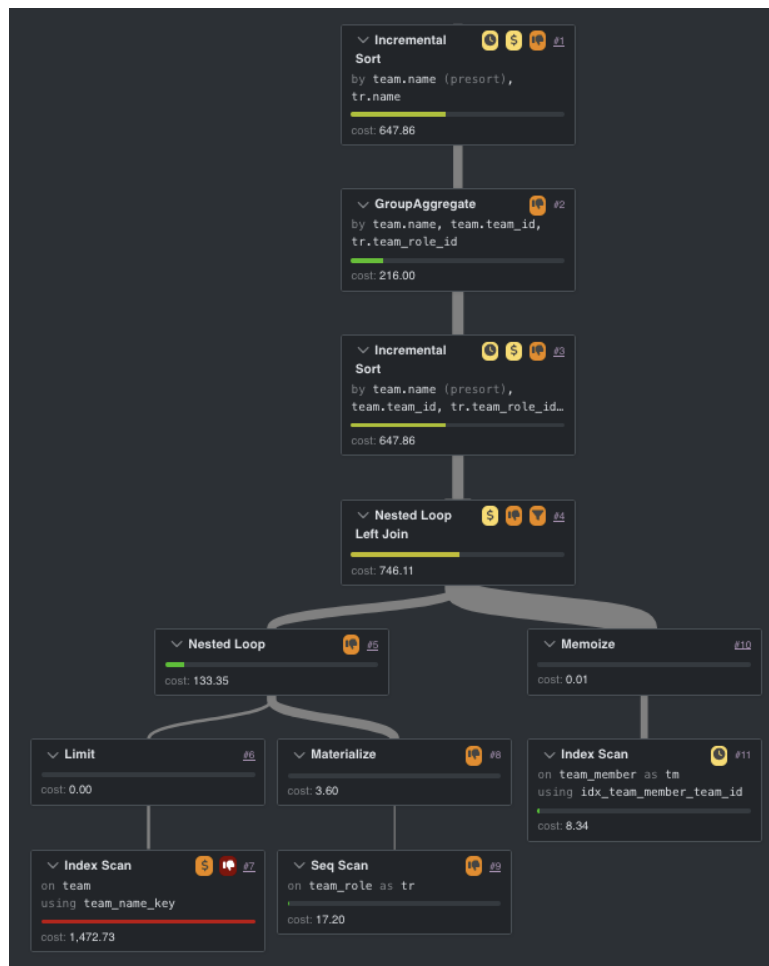


Рис. 26: План выполнения запроса 8

team_name	team_role	member_count
Bayes Falcons 100996	captain	0
Bayes Falcons 100996	member	0
Bayes Falcons 100996	капитан	0
Bayes Falcons 100996	участник	1
Bayes Falcons 102166	captain	0
Bayes Falcons 102166	member	1
Bayes Falcons 102166	капитан	0
Bayes Falcons 102166	участник	1
Bayes Falcons 118880	captain	0
Bayes Falcons 118880	member	1
Bayes Falcons 118880	капитан	0
Bayes Falcons 118880	участник	1
Bayes Falcons 118998	captain	1
Bayes Falcons 118998	member	0
Bayes Falcons 118998	капитан	1
Bayes Falcons 118998	участник	0
Bayes Falcons 121654	captain	0
Bayes Falcons 121654	member	1
Bayes Falcons 121654	капитан	0
Bayes Falcons 121654	участник	0
Bayes Falcons 121995	captain	0
Bayes Falcons 121995	member	1
Bayes Falcons 121995	капитан	0
Bayes Falcons 121995	участник	3
Bayes Falcons 123038	captain	0
Bayes Falcons 123038	member	1
Bayes Falcons 123038	капитан	2

Рис. 27: Результат запроса 8: число участников по командам и ролям

участники команд по ролям (первые 15)

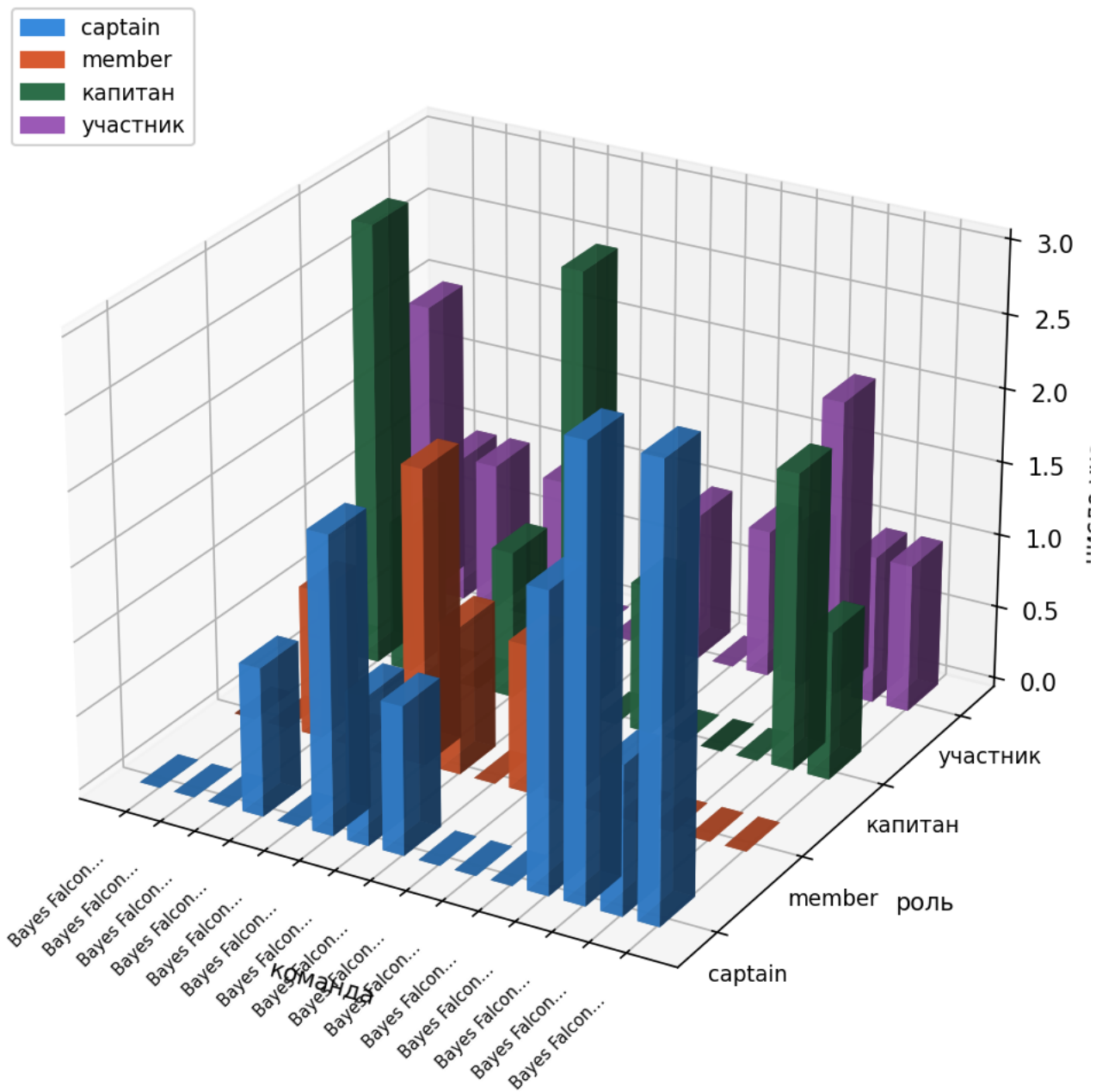


Рис. 28: Число участников команд по ролям (первые 15 команд)

Время выполнения запроса:

Planning Time: 3.783 ms

Execution Time: 87.782 ms

4.10 Запрос 9

Для команды A в соревновании B увеличить оценку случайной попытки на 10%.

```
1 UPDATE submission
2 SET metric_value = ROUND(metric_value * 1.1, 6)
3 WHERE submission_id = (
4     SELECT s.submission_id
5     FROM submission s
6         JOIN participation p ON p.participation_id =
7             s.participation_id
8         JOIN team t ON t.team_id = p.team_id
9         JOIN competition c ON c.competition_id =
10             p.competition_id
11     WHERE s.metric_value IS NOT NULL
12     ORDER BY RANDOM()
13     LIMIT 1
14 )
15 RETURNING submission_id, metric_value;
```

Реляционная формула запроса:

$$\text{UPDATE}_{\text{metric_value} \leftarrow \text{ROUND}(\text{metric_value} \times 1.1, 6)} \left(\text{submission}, \right.$$
$$\left. \sigma_{\text{submission_id} = \text{sid}^*}(\text{submission}) \right)$$

где $\text{sid}^* = \pi_{\text{submission_id}}^{\text{LIMIT } 1} \left(\right.$

$$\left. \tau_{\text{RANDOM}()} \left(\right.$$
$$\left. \begin{array}{l} \sigma_{\text{metric_value} \neq \text{NULL}}(\text{submission}) \\ \bowtie \text{ participation} \\ \bowtie \text{ team} \\ \bowtie \text{ competition} \end{array} \right)$$
$$\left. \right)$$

Подзапрос (InitPlan, возвращает \$0) выбирает одну случайную попытку с непустой оценкой. Планировщик строит цепочку из трёх Hash Join: Seq Scan по submission (cost 3188.43) фильтрует строки с metric_value IS NOT NULL и соединяется с Seq Scan по participation (cost 960.31) по ключу participation_id (cost 1040.15); результат объединяется с Seq Scan по team (cost 357.98) по team_id (cost 551.03); наконец, присоединяется Seq Scan по competition (cost 193.67) по competition_id (cost 694.60). Полученный набор сортируется по RANDOM() с применением top-N heapsort (cost 877.82),

после чего `Limit` извлекает одну строку. Полученный `submission_id` передаётся как константа `$0`; основной запрос находит строку через `Index Scan` по `submission_pkey` (cost 8.32) и выполняет `Update` с `RETURNING`.

Таблица 32: Последовательность выполнения запроса 9

Шаг	Операция	Описание
1	Seq Scan (submission s)	Чтение попыток с фильтром <code>metric_value IS NOT NULL</code> , cost 3 188.43
2	Seq Scan (participation)	Полное чтение участников, cost 960.31
3	Hash Join (participation)	Соединение попыток с участниками по <code>participation_id</code> , cost 1 040.15
4	Seq Scan (team)	Полное чтение команд, cost 357.98
5	Hash Join (team)	Соединение с командами по <code>team_id</code> , cost 551.03
6	Seq Scan (competition)	Полное чтение соревнований, cost 193.67
7	Hash Join (competition)	Соединение с соревнованиями по <code>competition_id</code> , cost 694.60
8	Sort (RANDOM())	Случайная сортировка, <code>top-N heapsort</code> , cost 877.82
9	Limit	Выбор одной строки
10	Index Scan (submission_pkey)	Поиск строки для обновления по первичному ключу, cost 8.32
11	Update + RETURNING	Обновление <code>metric_value</code> , возврат строки

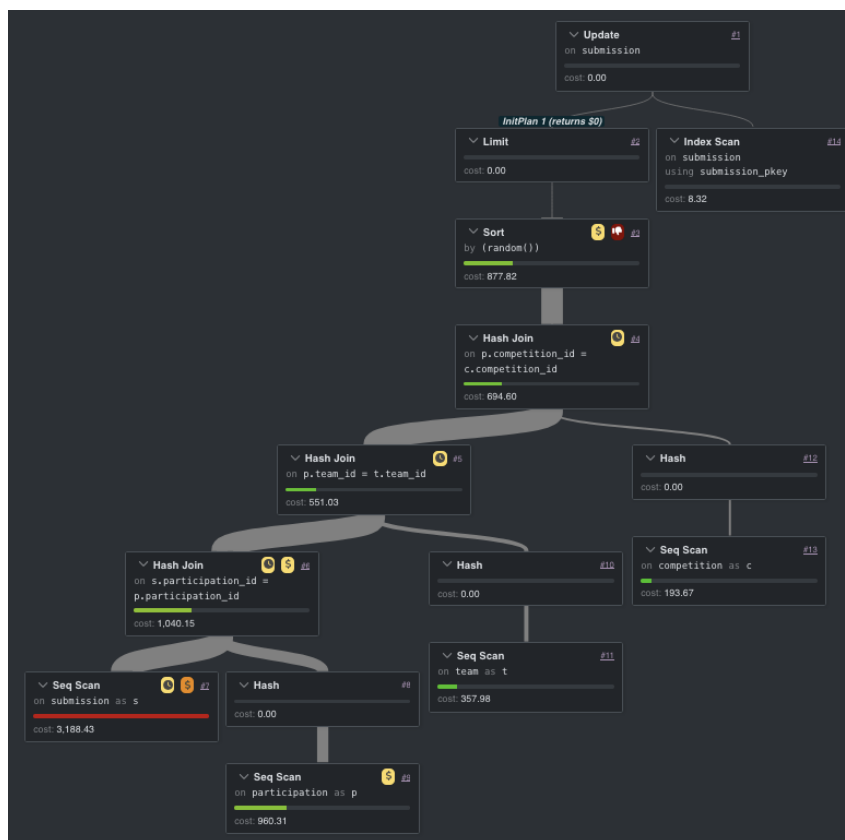


Рис. 29: План выполнения запроса 9

```
submission_id | metric_value
-----+-----
          19720 |      0.382235
(1 row)

UPDATE 1
```

Рис. 30: Результат запроса 9: обновлённая оценка попытки

Время выполнения запроса:

Planning Time: 4.890 ms

Execution Time: 53.868 ms

Заключение

В ходе работы был выполнен анализ предметной области «Соревнования по машинному обучению», определены цели функционирования базы данных, выявлены сущности и их атрибуты. На основе анализа построены ER-диаграмма в нотации Чена и схема базы данных, описаны таблицы с обоснованием типов атрибутов.

Схема реализована в СУБД PostgreSQL. Для заполнения базы данных написана программа на языке Python с использованием библиотек **asyncpg** и **Faker**. В итоговой базе содержится 317614 записей в 19 таблицах; основные транзакционные таблицы — **submission** (117043 строк), **participation** (58631 строк) и **team_member** (58228 строк).

Выполнены 9 запросов к базе данных, запросы 1 и 6 представлены в двух вариантах реализации. Для каждого варианта приведены реляционная формула, план выполнения, последовательность операций планировщика и время выполнения; проведён анализ стоимостного профиля узлов плана с указанием доминирующих операций и обоснованием выбора планировщиком конкретных методов соединения и доступа к данным. По результатам сравнения установлено, что использование **InitPlan** с вычислением подзапроса однократно предпочтительнее конструкций **>= ALL** и декартова произведения с агрегатом в ключах **GROUP BY**.

Работа выполнялась на устройстве Apple MacBook Air с процессором Apple M3 и 16 ГБ оперативной памяти. СУБД PostgreSQL запускалась в Docker-контейнере на образе **postgres:16-alpine**. Генерация данных реализована на языке Python с использованием библиотек **asyncpg** и **Faker**, визуализация результатов выполнена средствами **matplotlib**. Среда разработки — Visual Studio Code.

Список литературы

- [1] explain.dalibo.com [Электронный ресурс]: Dalibo. — URL: <https://explain.dalibo.com> (Дата обращения: 06.05.2026).
- [2] Попов С. Г. Лист самостоятельной проверки хода выполнения курсовой работы по дисциплине «Теоретические основы баз данных». 4 семестр.
- [3] Соревнования по машинному обучению [Электронный ресурс]: Kaggle. — URL: <https://www.kaggle.com/competitions> (Дата обращения: 06.05.2026).
- [4] Система управления базами данных [Электронный ресурс]: Википедия. — URL: https://ru.wikipedia.org/wiki/РҒРҫCFCTРхРёРө_СТР»СТРөРӱРьРхР,,РҫCS_РҫРөРҫРөРёРҫ_РұРөР,,Р,,СКСЖ (Дата обращения: 06.05.2026).
- [5] Машинное обучение [Электронный ресурс]: Википедия. — URL: https://ru.wikipedia.org/wiki/РЁРөСІРҫР,,Р,,Р«Рх_Р«РҫСТҢСЬРхР,,РҫРх (Дата обращения: 06.05.2026).
- [6] PostgreSQL 16 Documentation [Электронный ресурс]: The PostgreSQL Global Development Group. — URL: <https://www.postgresql.org/docs/16/> (Дата обращения: 06.05.2026).
- [7] asyncpg Documentation [Электронный ресурс]: MagicStack Inc. — URL: <https://magicstack.github.io/asyncpg/current/> (Дата обращения: 06.05.2026).
- [8] Takahashi M., Azuma S., Trend C. L. The Manga Guide to Databases. — No Starch Press, 2009. — 214 p.
- [9] Ramakrishnan R., Gehrke J. Database Management Systems. — 3rd ed. — McGraw-Hill, 2003. — 1065 p.